

ui.audio 全面详细阐述

`ui.audio` 是 NiceGUI 框架中用于**音频播放**的核心组件，基于 HTML5 原生元素封装，支持主流音频格式、自定义控制样式、播放状态监听及编程式控制，适用于网页音频播放、背景音乐、语音文件预览等场景。该组件轻量化集成，无需额外依赖，同时提供与 NiceGUI 生态一致的 API 设计，支持响应式配置与事件绑定。以下从核心特性、使用场景、配置选项、事件系统、进阶用法等方面展开详细说明。

一、核心概述

1. 功能定位

- 基础能力：加载并播放本地 / 网络音频文件，支持暂停、音量调节、进度控制等核心播放功能；
- 格式支持：兼容 HTML5 原生标准格式（MP3、WAV、OGG、AAC 等）；
- 样式定制：支持原生控件样式、自定义控件（通过事件与 UI 组件联动）；
- 状态监听：支持播放、暂停、结束、加载完成等事件，可实时获取播放状态；
- 编程控制：支持通过代码触发播放、暂停、跳转进度、调节音量等操作。

2. 核心优势

优势点	说明
轻量化集成	基于原生 HTML5 音频 API，无需额外依赖（如 Flash、第三方播放器库）；
多源支持	支持网络 URL、本地文件、Base64 编码等多种音频源；
样式灵活	可使用原生控件，也可隐藏原生控件并通过 NiceGUI 组件自定义播放面板；
状态与事件完备	提供完整的播放状态回调（播放 / 暂停 / 结束 / 加载失败等），支持精准控制；
响应式适配	支持 Tailwind/Quasar 样式类，可适配不同屏幕尺寸的布局需求；
生态兼容	可与 NiceGUI 其他组件（如按钮、滑块、进度条）无缝联动，构建复杂音频交互场景。

3. 支持的音频格式

格式	扩展名	浏览器兼容性	适用场景
MP3	<code>.mp3</code>	所有现代浏览器 (Chrome/Firefox/Edge/Safari)	最常用格式，适合音乐、语音文件
WAV	<code>.wav</code>	所有现代浏览器	无损音频，适合短音效、语音片段
OGG	<code>.ogg</code> / <code>.oga</code>	Chrome/Firefox/Edge，不支持 Safari	开源格式，压缩比高，适合网页音频
AAC	<code>.aac</code>	所有现代浏览器	苹果生态首选格式，音质优于 MP3

格式	扩展名	浏览器兼容性	适用场景
WebM	.webm	Chrome/Firefox/Edge, 部分支持 Safari	开源格式, 适合流媒体音频

注：具体格式兼容性需参考目标浏览器，建议优先使用 MP3 格式以保证最大兼容性。

二、基础使用与音频源支持

`ui.audio` 的核心参数为 `source`，支持多种音频源输入，用法简洁直观。以下是不同音频源的使用示例：

1. 网络音频（URL 源）

直接通过网络 URL 加载音频文件，适用于公开音频资源（如 CDN 上的音乐、语音文件）。

```
from nicegui import ui

# 加载网络 MP3 音频（示例为 NiceGUI 官方测试音频）
ui.audio('https://nicegui.io/examples/hello.mp3')

ui.run()
```

2. 本地音频文件

通过本地文件路径加载音频，需注意文件路径的正确性（建议使用相对路径）。

```
from nicegui import ui

# 加载本地音频文件（相对路径，以入口文件 main.py 为基准）
ui.audio('static/audio/background.mp3')

# 加载绝对路径音频（需确保运行环境有权限访问）
# ui.audio('/home/user/audio/sound.wav')

ui.run()
```

- 路径说明：
 - 相对路径：推荐将音频文件放在 `static` 文件夹下（NiceGUI 默认静态资源目录），直接通过 `static/[文件夹]/[文件名]` 访问；
 - 绝对路径：适用于固定路径的音频文件，但跨环境兼容性较差，不推荐优先使用。

3. Base64 编码音频

将音频文件编码为 Base64 字符串直接嵌入，无需额外文件依赖，适用于小型音频（如短音效、提示音）。

```
from nicegui import ui
import base64

# 读取本地音频文件并编码为 Base64
def audio_to_base64(file_path):
    with open(file_path, 'rb') as f:
```

```
base64_data = base64.b64encode(f.read()).decode('utf-8')
# 前缀格式: data:audio/[格式];base64,
return f'data:audio/mp3;base64,{base64_data}'

# 转换本地 MP3 为 Base64 并加载
base64_audio = audio_to_base64('static/audio/alert.mp3')
ui.audio(base64_audio)

ui.run()
```

- **格式要求:** Base64 字符串需添加前缀 `data:audio/[格式];base64`, (如 MP3 对应 `data:audio/mp3;base64`), 格式需与音频文件一致。

4. 响应式音频源（动态切换）

结合 NiceGUI 的响应式机制，实现音频源的动态切换（如通过按钮切换不同音乐）。

```
from nicegui import ui

# 响应式变量存储音频源
audio_source = ui.reactive('https://nicegui.io/examples/hello.mp3')

# 音频组件绑定响应式源
audio = ui.audio().bind_source(audio_source)

# 按钮切换音频源
ui.row([
    ui.button('播放音频1', on_click=lambda: setattr(audio_source, 'value',
'https://nicegui.io/examples/hello.mp3')),
    ui.button('播放音频2', on_click=lambda: setattr(audio_source, 'value',
'https://example.com/another-audio.mp3')),
])

ui.run()
```

三、核心配置选项

`ui.audio` 提供丰富的配置属性，用于控制音频播放行为、控件样式、响应式状态等，核心属性如下：

属性名	类型	说明	默认值
<code>source</code>	<code>BindableProperty</code>	音频源（支持 URL、本地路径、Base64 字符串），可绑定响应式变量；	<code>None</code>
<code>autoplay</code>	<code>bool</code>	音频加载完成后是否自动播放（部分浏览器限制：需用户交互后才能自动播放）；	<code>False</code>
<code>loop</code>	<code>bool</code>	是否循环播放音频；	<code>False</code>
<code>muted</code>	<code>bool</code>	是否默认静音；	<code>False</code>

属性名	类型	说明	默认值
<code>controls</code>	<code>bool</code>	是否显示原生播放控件（播放 / 暂停、进度条、音量调节）；	<code>True</code>
<code>preload</code>	<code>str</code>	预加载策略：' <code>none</code> '（不预加载）、' <code>metadata</code> '（仅预加载元数据）、' <code>auto</code> '（自动预加载）；	<code>'auto'</code>
<code>volume</code>	<code>float</code>	初始音量（0.0~1.0, 0 为静音, 1 为最大音量）；	<code>1.0</code>
<code>classes</code>	<code>Classes[Self]</code>	应用于音频元素的 CSS 类（支持 Tailwind、Quasar 类）；	<code>''</code>
<code>style</code>	<code>Style[Self]</code>	内联 CSS 样式（如 <code>width: 100%;</code> ）；	<code>''</code>
<code>html_id</code>	<code>str</code>	HTML DOM 元素 ID（v2.16.0+ 支持）；	<code>''</code>
<code>visible</code>	<code>BindableProperty</code>	组件是否可见（可绑定响应式变量）；	<code>True</code>

配置示例（自定义播放行为）

```
from nicegui import ui

# 配置：自动播放（需用户交互后生效）、循环播放、默认静音、隐藏原生控件
ui.audio(
    source='https://nicegui.io/examples/hello.mp3',
    autoplay=True,
    loop=True,
    muted=True,
    controls=False,
    preload='metadata', # 仅预加载音频时长等元数据
    volume=0.7, # 初始音量 70%
).classes('w-full') # 宽度 100% 适配父容器

ui.run()
```

四、核心事件系统

`ui.audio` 支持完整的音频播放状态事件，可实时监听播放、暂停、结束、加载等状态变化，事件回调中可获取当前播放进度、音量等关键信息。

1. 支持的事件类型与参数

事件名	触发时机	事件参数 (e) 关键属性
<code>play</code>	音频开始播放时触发（包括首次播放、暂停后恢复）	<code>current_time</code> ：当前播放时间（秒）； <code>duration</code> ：音频总时长（秒）； <code>volume</code> ：当前音量；
<code>pause</code>	音频暂停时触发	同 <code>play</code>

事件名	触发时机	事件参数 (e) 关键属性
<code>ended</code>	音频播放结束时触发（非循环模式下）	同 <code>play</code>
<code>loadeddata</code>	音频数据加载完成（可播放）时触发	<code>duration</code> ：音频总时长（秒）； <code>buffered</code> ：已缓冲的时间范围（数组）；
<code>timeupdate</code>	播放进度更新时触发（约每秒 4 次）	<code>current_time</code> ：当前播放时间（秒）； <code>progress</code> ：播放进度（0.0~1.0）；
<code>volumechange</code>	音量变化时触发（包括静音切换）	<code>volume</code> ：当前音量（0.0~1.0）； <code>muted</code> ：是否静音；
<code>error</code>	音频加载或播放失败时触发	<code>error</code> ：错误信息对象（包含错误代码和描述）；
<code>canplay</code>	音频可开始播放（缓冲足够）时触发	同 <code>loadeddata</code>
<code>waiting</code>	音频缓冲中（暂时无法播放）时触发	无额外参数

2. 事件绑定示例（核心场景）

示例 1：监听播放状态与进度

```
from nicegui import ui

audio = ui.audio('https://nicegui.io/examples/hello.mp3').classes('w-full')

# 播放状态显示
status_label = ui.label('状态：未播放')
# 进度显示
progress_label = ui.label('进度：0%')

# 监听播放事件
@audio.on('play')
def on_play(e):
    status_label.set_text(f'状态：播放中 | 总时长：{e.duration:.1f}秒')

# 监听暂停事件
@audio.on('pause')
def on_pause(e):
    status_label.set_text(f'状态：已暂停 | 当前进度：{e.current_time:.1f}秒')

# 监听播放结束事件
@audio.on('ended')
def on_ended(e):
    status_label.set_text('状态：播放结束')

# 监听进度更新事件
@audio.on('timeupdate')
```

```
def on_timeupdate(e):
    progress = (e.current_time / e.duration) * 100 if e.duration else 0
    progress_label.set_text(f'进度: {progress:.1f}%')

# 监听加载失败事件
@audio.on('error')
def on_error(e):
    status_label.set_text(f'状态: 加载失败 | 错误: {e.error}')

ui.run()
```

示例 2: 音量变化监听

```
from nicegui import ui

audio = ui.audio('https://nicegui.io/examples/hello.mp3', volume=0.5).classes('w-full')

volume_label = ui.label('当前音量: 50%')

# 监听音量变化
@audio.on('volumechange')
def on_volume_change(e):
    volume_percent = e.volume * 100
    volume_label.set_text(f'当前音量: {volume_percent:.0f}% | 静音: {"是" if e.muted else "否"}')

ui.run()
```

五、程式控制方法

`ui.audio` 提供一系列方法用于通过代码控制音频播放行为，支持播放、暂停、跳转进度、调节音量等操作，核心方法如下：

方法名	作用	参数说明
<code>play()</code>	开始播放音频（若已暂停则恢复）	无参数；返回 <code>None</code> （异步执行，无需等待）
<code>pause()</code>	暂停当前播放	无参数；返回 <code>None</code>
<code>stop()</code>	停止播放并重置进度到起始位置（0 秒）	无参数；返回 <code>None</code>
<code>set_volume(volume)</code>	设置音量（0.0~1.0）	<code>volume</code> ：目标音量（浮点数，超出范围会被自动截断）；返回 <code>None</code>
<code>set_muted(muted)</code>	设置是否静音	<code>muted</code> ： <code>True</code> （静音）/ <code>False</code> （取消静音）；返回 <code>None</code>
<code>seek(seconds)</code>	跳转播放进度到指定时间点	<code>seconds</code> ：目标时间（秒）；超出音频时长则跳转到结束位置；返回 <code>None</code>

方法名	作用	参数说明
<code>get_current_time()</code>	获取当前播放时间（秒）	无参数；返回 <code>float</code> 类型
<code>get_duration()</code>	获取音频总时长（秒）（需加载完成后调用，否则返回 0）	无参数；返回 <code>float</code> 类型
<code>get_volume()</code>	获取当前音量（0.0~1.0）	无参数；返回 <code>float</code> 类型
<code>is_playing()</code>	判断音频是否正在播放	无参数；返回 <code>bool</code> 类型
<code>is_muted()</code>	判断音频是否静音	无参数；返回 <code>bool</code> 类型
<code>set_source(source)</code>	切换音频源	<code>source</code> ：新音频源（URL / 本地路径 / Base64）；返回 <code>None</code>
<code>force_reload()</code>	强制重新加载音频源	无参数；返回 <code>None</code>

方法使用示例（自定义播放面板）

结合 NiceGUI 组件（按钮、滑块、进度条）实现自定义音频播放面板，替代原生控件：

```
from nicegui import ui

class CustomAudioPlayer:
    def __init__(self, audio_source):
        self.source = audio_source
        self.audio = None
        self.duration = 0 # 音频总时长（秒）
        self._build_ui()

    def _build_ui(self):
        # 音频组件（隐藏原生控件）
        self.audio = ui.audio(self.source, controls=False).classes('w-full')

        # 自定义控制面板
        with ui.row().classes('items-center gap-4 mt-2'):
            # 播放/暂停按钮
            self.play_btn = ui.button('▶ 播放', on_click=self.toggle_play)

            # 进度条（绑定播放进度）
            self.progress_bar = ui.slider(min=0, max=100, value=0, step=0.1)
            self.progress_bar.on('change', self.seek_to_progress)

            # 音量滑块
            self.volume_slider = ui.slider(min=0, max=1, value=0.7, step=0.01).classes('w-32')
            self.volume_slider.on('change', lambda e: self.audio.set_volume(e.value))

            # 静音按钮
            self.mute_btn = ui.button('🔊', on_click=self.toggle_mute)

            # 时间显示
```

```

        self.time_label = ui.label('00:00 / 00:00').classes('w-24 text-center')

# 绑定音频事件
self._bind_events()

def _bind_events(self):
    # 加载完成：获取总时长
    @self.audio.on('loadeddata')
    def on_loaded(e):
        self.duration = e.duration
        self.progress_bar.max = self.duration
        self.time_label.set_text(self._format_time(0) + ' / ' +
self._format_time(self.duration))

    # 进度更新：同步进度条和时间显示
    @self.audio.on('timeupdate')
    def on_timeupdate(e):
        current_time = e.current_time
        self.progress_bar.set_value(current_time)
        self.time_label.set_text(self._format_time(current_time) + ' / ' +
self._format_time(self.duration))

    # 播放状态更新：切换按钮图标
    @self.audio.on('play')
    def on_play(e):
        self.play_btn.set_text('⏸ 暂停')

    @self.audio.on('pause')
    def on_pause(e):
        self.play_btn.set_text('▶ 播放')

    # 音量变化：同步音量滑块
    @self.audio.on('volumechange')
    def on_volume_change(e):
        self.volume_slider.set_value(e.volume)
        self.mute_btn.set_text('🔊' if e.muted else '🔇')

def toggle_play(self):
    # 切换播放/暂停
    if self.audio.is_playing():
        self.audio.pause()
    else:
        self.audio.play()

def toggle_mute(self):
    # 切换静音
    current_mute = self.audio.is_muted()
    self.audio.set_muted(not current_mute)

def seek_to_progress(self, e):
    # 进度条跳转
    target_time = e.value
    self.audio.seek(target_time)

```

```

    @staticmethod
    def _format_time(seconds):
        # 格式化时间: 秒 → 分:秒 (如 125 秒 → 02:05)
        minutes = int(seconds // 60)
        secs = int(seconds % 60)
        return f'{minutes:02d}:{secs:02d}'

# 初始化自定义音频播放器
player = CustomAudioPlayer('https://nicegui.io/examples/hello.mp3')

ui.run()

```

六、进阶用法

场景 1：背景音乐（自动播放 + 全局控制）

实现网页背景音乐功能，支持全局开关、音量调节，且不影响其他音频播放：

```

from nicegui import ui

# 全局响应式变量：控制背景音乐开关和音量
bgm_enabled = ui.reactive(True)
bgm_volume = ui.reactive(0.3)

# 背景音乐组件（隐藏控件、自动播放、循环）
bgm_audio = ui.audio(
    source='https://nicegui.io/examples/hello.mp3',
    autoplay=True,
    loop=True,
    controls=False,
    volume=bgm_volume.value,
).classes('hidden') # 隐藏组件（仅后台播放）

# 绑定响应式变量
bgm_audio.bind_volume(bgm_volume)
bgm_audio.bind_visible(bgm_enabled)

# 控制面板
with ui.card().classes('fixed top-4 right-4 z-10'):
    ui.label('背景音乐').classes('font-bold')
    # 开关
    ui.switch('启用', value=bgm_enabled.value).bind_value(bgm_enabled)
    # 音量滑块
    ui.slider('音量', min=0, max=1, value=bgm_volume.value).bind_value(bgm_volume)

# 其他内容（示例）
ui.label('网页内容').classes('text-2xl text-center mt-20')

ui.run()

```

场景 2：音频列表切换播放

实现多音频列表，支持点击切换、上一曲 / 下一曲功能：

```
from nicegui import ui

# 音频列表（URL 或本地路径）
audio_list = [
    {'title': '音频1', 'source': 'https://nicegui.io/examples/hello.mp3'},
    {'title': '音频2', 'source': 'https://example.com/audio2.mp3'},
    {'title': '音频3', 'source': 'https://example.com/audio3.mp3'},
]

current_index = ui.reactive(0) # 当前播放索引

# 音频组件
audio = ui.audio(audio_list[current_index.value]['source']).classes('w-full')

# 控制按钮
with ui.row().classes('mt-4 gap-2'):
    def prev_audio():
        # 上一曲
        new_index = (current_index.value - 1) % len(audio_list)
        current_index.set_value(new_index)
        audio.set_source(audio_list[new_index]['source'])
        audio.play()

    def next_audio():
        # 下一曲
        new_index = (current_index.value + 1) % len(audio_list)
        current_index.set_value(new_index)
        audio.set_source(audio_list[new_index]['source'])
        audio.play()

    ui.button('上一曲', on_click=prev_audio)
    ui.button('播放/暂停', on_click=lambda: audio.play() if not audio.is_playing() else
audio.pause())
    ui.button('下一曲', on_click=next_audio)

# 音频列表选择
ui.label('音频列表').classes('mt-4 font-bold')
for i, item in enumerate(audio_list):
    ui.button(
        item['title'],
        on_click=lambda idx=i: (current_index.set_value(idx),
audio.set_source(audio_list[idx]['source']), audio.play()),
        classes='w-full mt-1'
    ).bind_classes({'bg-blue-500 text-white': current_index.value == i})

ui.run()
```

场景 3：结合录音组件实现音频预览

与 `ui.input` 或第三方录音库配合，实现录音后即时预览功能（示例使用 `sounddevice` 库录音）：

```
from nicegui import ui
import sounddevice as sd
import soundfile as sf
import numpy as np
import tempfile
import os

class AudioRecorder:
    def __init__(self):
        self.recording = False
        self.frames = []
        self.sample_rate = 44100
        self.temp_file = None
        self._build_ui()

    def _build_ui(self):
        ui.label('录音与预览').classes('text-2xl font-bold')

        # 录音控制按钮
        self.record_btn = ui.button('开始录音', on_click=self.toggle_recording).classes('bg-red-500 text-white')

        # 音频预览组件（初始隐藏）
        self.preview_audio = ui.audio().classes('w-full mt-2 hidden')

    def toggle_recording(self):
        if not self.recording:
            # 开始录音
            self.recording = True
            self.frames = []
            self.record_btn.set_text('停止录音')
            self.record_btn.classes('bg-green-500 text-white')
            # 启动录音流
            self.stream = sd.InputStream(samplerate=self.sample_rate, channels=1,
callback=self._record_callback)
            self.stream.start()
        else:
            # 停止录音
            self.recording = False
            self.stream.stop()
            self.stream.close()
            self.record_btn.set_text('开始录音')
            self.record_btn.classes('bg-red-500 text-white')
            # 保存为临时文件并预览
            self._save_and_preview()

    def _record_callback(self, indata, frames, time, status):
        # 录音回调：收集音频数据
        self.frames.append(indata.copy())
```

```
def _save_and_preview(self):
    # 保存录音到临时文件
    if self.temp_file:
        os.unlink(self.temp_file) # 删除旧临时文件
    self.temp_file = tempfile.NamedTemporaryFile(suffix='.wav', delete=False).name
    # 合并音频数据并保存
    audio_data = np.concatenate(self.frames, axis=0)
    sf.write(self.temp_file, audio_data, self.sample_rate)
    # 预览录音
    self.preview_audio.set_source(self.temp_file)
    self.preview_audio.classes('w-full mt-2') # 显示组件

# 依赖安装提示：需安装 sounddevice 和 soundfile
ui.label('提示：需安装依赖: pip install sounddevice soundfile').classes('text-red-500')

# 初始化录音器
recorder = AudioRecorder()

ui.run()
```

七、注意事项与优化建议

1. 自动播放限制

- 现代浏览器为提升用户体验，限制音频自动播放（`autoplay=True`）：需用户先进行交互（如点击页面、输入）后，自动播放才能生效；
- 解决方案：在页面添加“开始播放”按钮，用户点击后通过 `audio.play()` 触发播放（如背景音乐场景）。

2. 音频加载与缓冲

- 网络音频加载较慢时，可通过 `preload='metadata'` 仅预加载时长等元数据，减少初始加载时间；
- 长音频文件建议使用流式传输（如 HLS 协议），但 `ui.audio` 原生不支持 HLS，需结合第三方库（如 `hls.js`）实现。

3. 跨域问题

- 若加载跨域音频文件（不同域名），需确保服务端设置 `Access-Control-Allow-Origin` 响应头，否则可能出现加载失败；
- 本地开发时，若使用 `ui.run()` 的默认配置，本地文件无跨域问题；部署时需注意服务器配置。

4. 性能优化

- 避免同时播放多个音频文件（可能导致音质下降或卡顿）；
- 不需要的音频组件及时调用 `delete()` 方法销毁，释放资源；
- 短音频（如音效）建议使用 Base64 编码嵌入，减少网络请求。

5. 兼容性处理

- 针对老旧浏览器（如 IE），可提供音频下载链接作为降级方案；
- 若需支持特殊音频格式（如 FLAC），可先转换为 MP3 或 WAV 格式，或使用第三方解码库。

6. 移动端适配

- 移动端浏览器的原生音频控件样式可能与桌面端不同，建议使用自定义控件（如示例中的 `CustomAudioPlayer`）保证跨端一致性；
- 移动端需注意音频播放时的锁屏控制（如显示播放状态、进度条），可通过 HTML5 `MediaSession` API 实现（需额外编写 JavaScript）。

总结

`ui.audio` 是 NiceGUI 中功能完备、易用性强的音频播放组件，基于 HTML5 原生音频 API 封装，支持多源音频加载、原生 / 自定义控件、播放状态监听及程式化控制。其核心优势在于轻量化集成与 NiceGUI 生态的无缝兼容，可快速实现从简单音频播放到复杂交互面板（如自定义播放器、音频列表）的各类场景。

掌握该组件的关键在于：

1. 理解音频源的多种输入方式（URL / 本地 / Base64）；
2. 熟练运用配置属性控制播放行为（自动播放、循环、静音等）；
3. 结合事件系统与程式化方法，实现自定义交互逻辑；
4. 规避浏览器限制（如自动播放、跨域）与兼容性问题。